BÁO CÁO THỰC HÀNH

APACHE SPARK

Mô hình Master-Worker với Docker trên Windows

Bài 01 | Lời giải 25 bài | 30/10/2026

Hồ sơ gồm báo cáo, mã nguồn Scala/PySpark, Docker Compose, notebook, dữ liệu và bảng kết quả đối chiếu.

TRẠNG THÁI THỰC HIỆN

Đã soạn giải pháp cho 25 bài. Chưa thực chạy Spark/Docker trên máy này. Các số liệu trong báo cáo được tính để đối chiếu, không được trình bày là kết quả Spark. Ảnh chụp, benchmark và output notebook cần bổ sung sau khi chạy.

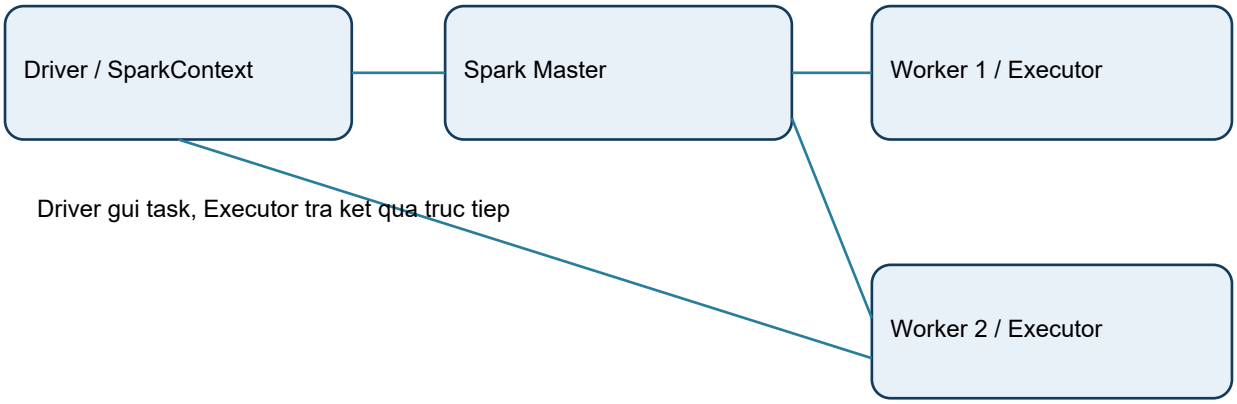
Mục lục và cách sử dụng

01. Kiến trúc Apache Spark
02. Kiểm tra Docker trên Windows
03. Xây dựng Spark Cluster
04. Quan sát Master và Worker
05. Tính toán RDD bằng Scala
06. Loại bỏ từ trùng lặp
07. WordCount Scala
08. Đọc và lọc CSV
09. Jupyter và PySpark
10. So sánh Python và Spark: 1 triệu số
11. Worker không đăng ký Master
12. WordCount PySpark và stopword
13. Kết hợp RDD và DataFrame
14. Gửi ứng dụng lên Cluster
15. Thống kê sales.csv
16. Tạo và quản lý Cluster mới
17. Thống kê điểm Scala
18. Danh sách từ PySpark
19. Thống kê điểm DataFrame
20. Hiệu năng với 5 triệu số
21. WordCount nhiều file
22. Thống kê orders.csv
23. Thử nghiệm Worker bị dừng
24. WordCount RDD và DataFrame trên Docker
25. Hệ thống dữ liệu cửa hàng

Mỗi bài bắt đầu ở trang mới. Mã nguồn chi tiết nằm trong thư mục apps; README.md chứa lệnh chạy và danh sách minh chứng còn thiếu.

BÀI 01

Kiến trúc Apache Spark



Apache Spark là bộ máy xử lý dữ liệu phân tán, hỗ trợ RDD, SQL/DataFrame, streaming và học máy. RDD là tập dữ liệu phân tán bất biến; các phép biến đổi được thực thi trễ, khi có action như count hoặc collect.

Trong chế độ Standalone, Master quản lý đăng ký Worker và cấp tài nguyên cho ứng dụng. Worker quản lý tiến trình Executor trên máy của mình. Driver tạo SparkSession/SparkContext, xây DAG, chia stage/task và lập lịch task đến Executor. Executor thực thi task, giữ cache và trả kết quả cho Driver. Master không trực tiếp chia từng job thành task.

Luồng: spark-submit khởi tạo Driver; Driver đăng ký ứng dụng với Master; Master cấp tài nguyên, Worker tạo Executor; Driver gửi task trực tiếp đến Executor; Executor đọc các partition, xử lý, shuffle khi cần và gửi kết quả/trạng thái về Driver.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 02

Kiểm tra Docker trên Windows

Lệnh: docker --version; docker run hello-world; docker ps; docker ps -a. docker ps chỉ liệt kê container đang chạy; docker ps -a liệt kê cả container đã dừng hoặc đã tạo nhưng chưa chạy. hello-world thoát sau khi in thông báo nên thường xuất hiện trong ps -a.

Máy soạn bài chưa tìm thấy Docker CLI. Vì vậy chưa có phiên bản Docker thực tế hoặc ảnh chạy hello-world. Sau khi cài Docker Desktop và bật WSL2, chạy Run-All.ps1 để ghi transcript; tự chụp màn hình terminal thành evidence/02-docker.png.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 03

Xây dựng Spark Cluster

File docker-compose.yml khai báo apache/spark:3.5.1, một Master và hai Worker; mỗi Worker cấp 1 core và 1 GiB bộ nhớ cho ứng dụng. Cấu hình dự kiến toàn cụm: 2 core, 2 GiB. Đây là tài nguyên cấu hình, chưa phải số liệu quan sát.

Chạy docker compose up -d rồi docker ps. Hai Worker đăng ký đến spark://spark-master:7077: spark-master là tên DNS nội bộ Compose, 7077 là cổng giao tiếp Spark. localhost trong Worker chỉ trỏ về chính Worker, không trỏ sang Master.

Các node cùng gắn ./data vào /data và ./apps vào /opt/spark-apps. Điều này giúp Executor đọc được file; chỉ tạo file trong Master mà không chia sẻ sang Worker có thể gây lỗi không tìm thấy dữ liệu.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 04

Quan sát Master và Worker

Mở http://localhost:8080 (Master), http://localhost:8081 (Worker 1), http://localhost:8082 (Worker 2). Master URL dự kiến spark://spark-master:7077. Hai Worker dự kiến ALIVE, mỗi Worker 1 core/1024 MiB; tổng 2 core/2048 MiB.

Run-All.ps1 ghi evidence/master-before.json và evidence/workers.csv từ giao diện JSON thực tế. Chưa có bảng quan sát hoặc ảnh Master trong môi trường hiện tại.

Nếu chỉ còn một Worker hoạt động, Master và cụm vẫn có thể nhận/chạy ứng dụng nếu còn đủ tài nguyên. Hiệu năng và khả năng song song giảm; không đảm bảo mọi ứng dụng đều chạy được nếu yêu cầu tài nguyên vượt phần còn lại.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 05

Tính toán RDD bằng Scala

Mã nguồn: apps/scala_exercises.scala, mục Bài 5. Tạo RDD từ List(12,15,20,25,30,35,40); gọi sum, min, max, mean, count.

Kết quả đối chiếu: tổng 177; nhỏ nhất 12; lớn nhất 40; trung bình 25.2857142857; số phần tử 7.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 06

Loại bỏ từ trùng lặp

Mã nguồn: apps/scala_exercises.scala, mục Bài 6. parallelize -> flatMap(_.split("\s+")) -> map(_.toLowerCase) -> distinct -> sortBy.

Kết quả đối chiếu: hadoop, java, python, scala, spark.

map biến mỗi câu thành một phần tử đầu ra (nếu split thì phần tử là mảng từ). flatMap vừa biến đổi vừa trải các mảng thành từng từ trong cùng RDD; vì vậy distinct áp dụng lên từng từ.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 07

WordCount Scala

Mã nguồn: apps/scala_exercises.scala, mục Bài 7. Tách từ, chuẩn hóa chữ thường, map thành (word,1), reduceByKey(_+_), sortByKey để kết quả dễ đối chiếu.

Kết quả đối chiếu: data=3; spark=3; big=2; is=1; powerful=1; processing=1; science=1; supports=1; with=1. Tổng 14 từ.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 08

Đọc và lọc CSV

Dữ liệu: data/students.csv; mã Scala trong scala_exercises.scala. Khai báo schema rõ để age/score là số; đọc header=true; show toàn bộ; lọc major == Data; lọc score >= 8.5; select name, major, score.

Nhóm Data: Binh (22,9.0), Dung (23,7.5). Điểm >=8.5: An (IT,8.5), Binh (Data,9.0), Hoa (IT,9.5). Phép chọn 3 cột thực hiện trên toàn bộ 5 sinh viên, độc lập với hai bộ lọc.

read.csv tạo DataFrame từ CSV; show kích hoạt việc lấy/in các dòng lên console; filter trả DataFrame chỉ gồm dòng thỏa điều kiện.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 09

Jupyter và PySpark

Notebook: notebooks/Bai_09.ipynb. Windows dùng Python 3.10/3.11, Temurin JDK 17, requirements.txt; đặt JAVA_HOME đến JDK 17, chạy python -m notebook từ thư mục bài.

SparkSession appName StudentPractice, master local[*]; RDD range(1,21); tổng 210, trung bình 10.5 (kết quả đối chiếu). Notebook chưa chạy, execution_count để null và không chèn output giả.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 10

So sánh Python và Spark: 1 triệu số

Mã nguồn: exercises.py --bai 10; kết quả tổng cả hai phương pháp phải là 500000500000. Dùng time.perf_counter cho độ phân giải cao. Đo Python sum(range), và Spark gồm tạo RDD + sum; loại trừ thời gian tạo SparkSession.

Chưa có thời gian thực chạy nên chưa xác định phương pháp nhanh hơn trong lần thử. Chương trình sẽ ghi results/benchmark_10.csv. Spark có chi phí JVM, lập lịch, chia partition, serialize và truyền dữ liệu; ở bài toán cộng số nhỏ, chi phí này có thể lớn hơn phần lợi từ song song.

Một lần thử không đủ kết luận Spark luôn nhanh hơn Python; cần kiểm soát dữ liệu, tài nguyên, số partition, cache, cold/warm run và lặp lại nhiều lần.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 11

Worker không đăng ký Master

Đây là phân tích tình huống, chưa phải nguyên nhân đã quan sát trên máy. Kiểm tra docker ps -a, docker logs spark-master, docker logs spark-worker-1, docker logs spark-worker-2, docker compose config.

Nếu log có UnknownHostException: kiểm tra tên service và mạng Compose. Nếu Connection refused: Master chưa sẵn sàng, sai cổng hoặc Master đã dừng. Nếu Worker thoát ngay: kiểm tra lệnh spark-class, bộ nhớ và lỗi entrypoint. Nếu dùng localhost:7077: sửa thành spark://spark-master:7077.

Sửa docker-compose.yml rồi docker compose up -d --force-recreate. Đợi Worker đăng ký; kiểm tra /json/ có hai Worker ALIVE và chụp Master sau sửa. Không kết luận nguyên nhân cụ thể nếu chưa có log.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 12

WordCount PySpark và stopword

Mã nguồn exercises.py --bai 12. Dùng regex [a-z0-9]+ để tách từ và bỏ dấu câu; đếm reduceByKey; sắp (-count,word) để kết quả ổn định khi đồng hạng.

Top 5 đối chiếu trước lọc: data=4, spark=4, big=3, distributed=2, for=2. Sau bỏ is, and, for, are: data=4, spark=4, big=3, distributed=2, a=1. Từ a vẫn giữ vì không thuộc danh sách stopword được yêu cầu.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 13

Kết hợp RDD và DataFrame

Mã nguồn apps/wordcount.py; dữ liệu data/documents gồm 5 file. RDD dùng textFile, flatMap, map, reduceByKey; DataFrame dùng read.text, regexp_replace, split, explode, groupBy/count. Cùng chuẩn hóa để so sánh công bằng.

Bảng đối chiếu đầy đủ: reference_results/bai13_wordcount.csv. Chương trình thực chạy sẽ assert hai kết quả bằng nhau và ghi results/wordcount/comparison, gồm word, rdd_count, df_count, equal.

RDD phù hợp xử lý phi cấu trúc và thuật toán tùy biến ở mức thấp. DataFrame thích hợp dữ liệu có schema, SQL, thống kê và tối ưu kế hoạch thực thi. RDD cần nhiều bước ánh xạ; DataFrame diễn tả thao tác theo cột và nhóm rõ hơn.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 14

Gửi ứng dụng lên Cluster

Dùng 5 tài liệu đã tạo (đáp ứng ít nhất 3). Chạy spark-submit bên trong spark-master, master spark://spark-master:7077 và deploy-mode client. Driver chạy trong container Master, Executor chạy tại Worker; đây vẫn là thực thi trên cụm.

Lệnh đầy đủ ở Run-All.ps1 đặt spark.driver.host=spark-master, bindAddress=0.0.0.0, executor.memory=512m và cores.max=2. Không hard-code local[*] trong script gửi lên cụm.

Master URL chỉ định cụm Standalone sẽ cấp tài nguyên; nếu dùng local thì không gửi tác vụ tới hai Worker. Quan sát Applications/Completed Applications tại Master; ảnh thực chạy chưa có.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 15

Thống kê sales.csv

Mã nguồn exercises.py --bai 15. Đọc schema rõ ràng; revenue = quantity * price; groupBy product/category, sum revenue, sắp doanh thu giảm dần; lưu chi tiết và hai bảng CSV.

Đối chiếu theo sản phẩm: Laptop 45.000.000; Monitor 9.000.000; Mouse 3.600.000; Keyboard 2.500.000 đồng. Theo danh mục: Electronics 54.000.000; Accessories 6.100.000. Tổng 60.100.000 đồng.

reference_results/bai15_*.csv là bảng tính đối chiếu. Thư mục kết quả Spark thật sẽ ở results/bai15_* với part-*.csv và _SUCCESS; kiểm tra bằng Get-ChildItem results/bai15_* -Recurse.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 16

Tạo và quản lý Cluster mới

Sao chép thư mục bài sang spark-cluster-homework; dừng cụm cũ trước vì tên container/cổng giống nhau. Dùng cùng file Compose đáp ứng một Master và hai Worker. Chạy up -d, kiểm tra ps và Master; stop rồi up -d, đợi đăng ký lại và kiểm tra.

docker compose up -d tạo/khởi động service ở nền, áp dụng cấu hình khi cần tái tạo. stop chỉ dừng container và giữ lại cấu hình/container. down dừng và xóa container cùng mạng mặc định; bind mount trên máy vẫn còn.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 17

Thống kê điểm Scala

Mã nguồn scala_exercises.scala, mục Bài 17. RDD chứa đúng 15 điểm đề bài; sum, mean, min, max, filter/count; tỷ lệ lấy số đạt chia tổng số.

Kết quả đối chiếu: tổng 114; trung bình 7.6; cao nhất 10; thấp nhất 5; đạt >=8 có 8 sinh viên; dưới 5 có 0 sinh viên; tỷ lệ 8/15 x100 = 53.3333%.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 18

Danh sách từ PySpark

Notebook notebooks/Bai_18.ipynb; mã triển khai trong exercises.py --bai 18. Tổng số từ sau tách 22; số từ phân biệt 13.

Thứ tự tăng dần: and, big, data, easy, is, learn, powerful, python, science, spark, supports, to, with. Kết quả đối chiếu; notebook chưa chạy trong môi trường hiện tại.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 19

Thống kê điểm DataFrame

Dữ liệu data/student_scores.csv; notebook notebooks/Bai_19.ipynb. Lọc Toán >=8: An, Binh, Hoa, Lan. Lớp IT01: An, Chi, Hoa.

Điểm trung bình giảm dần (đồng điểm sắp id tăng): Hoa 9.5; Binh 8.5; Chi 8.0; Lan 8.0; An 7.5; Dung 6.5. Top 3: Hoa, Binh, Chi. Chi và Lan đồng hạng; dùng id để chọn top 3 ổn định.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 20

Hiệu năng với 5 triệu số

exercises.py --bai 20 chạy mỗi phương pháp 3 lần; ghi từng số đo và trung bình vào results/benchmark_20.csv. Kết quả hai bên phải bằng 12500002500000. Không điền thời gian khi chưa đo thực tế.

Thời gian có thể dao động do tải máy, JVM/JIT, Python worker khởi động, cache hệ điều hành và shuffle. Bài tính tổng ít công việc cho mỗi phần tử nên overhead Spark có thể chiếm ưu thế.

Yếu tố ảnh hưởng: số Worker/core, RAM, GC, số partition, mạng và đĩa, serialization, kích thước dữ liệu, skew, cache/persist. Lần đầu là cold job; ba lần đo không tự chứng minh lợi thế cho mọi tình huống.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 21

WordCount nhiều file

Có 5 file nội dung khác nhau trong data/documents. apps/wordcount.py đọc cả thư mục, chuẩn hóa bằng regex, đếm và sắp giảm dần; chạy bằng Run-All.ps1 trên cụm cho yêu cầu mở rộng.

Top 10 đối chiếu: spark=5, and=3, data=3, big=2, distributed=2, is=2, python=2, supports=2, are=1, computing=1. Bảng đầy đủ ở reference_results/bai21_wordcount.csv.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 22

Thống kê orders.csv

Mã nguồn exercises.py --bai 22; schema gồm order_id INT, customer/product STRING, quantity INT, price LONG. printSchema xác nhận cấu trúc; revenue dùng phép nhân số nguyên.

Theo khách hàng: An 21.000.000; Dung 15.000.000; Binh 4.000.000; Chi 1.600.000 đồng. Tổng số lượng 15; tổng doanh thu 41.600.000 đồng.

Sản phẩm doanh thu cao nhất: Laptop 30.000.000; khách mua nhiều giá trị nhất: An 21.000.000. Monitor 9.000.000; Mouse 1.600.000; Keyboard 1.000.000. Laptop chiếm khoảng 72.12% doanh thu, nên theo dõi tồn kho; mẫu chỉ 7 đơn nên nhận xét có phạm vi hẹp.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 23

Thử nghiệm Worker bị dừng

Worker-Failure.ps1 lưu ba snapshot trước/trong/sau khi dừng Worker 1. Thời gian Master phát hiện có độ trễ; script đợi 65 giây. Trạng thái dự kiến: 2 ALIVE -> 1 ALIVE -> 2 ALIVE. Worker cũ có thể vẫn xuất hiện DEAD trước khi dọn lịch sử.

Master tiếp tục hoạt động. Với cấu hình này, mất một Worker làm giảm 1 core và 1 GiB tài nguyên được cấp. Executor trên Worker mất theo; Driver có thể lập lịch lại task mất trên Executor còn sống, tính lại partition từ lineage và fetch/recompute dữ liệu shuffle nếu cần.

Việc phục hồi phụ thuộc số lần retry, nguồn dữ liệu còn truy cập được và tài nguyên còn lại. Bổ sung Worker tăng khả năng song song cho task/ứng dụng phù hợp; không bảo đảm tăng tốc tuyến tính. Chưa tiến hành thử nghiệm thật.

Để quan sát mất Worker khi job đang chạy: mở terminal A chạy lại wordcount hoặc job dài, mở terminal B chạy Worker-Failure.ps1 khi job còn RUNNING. Job nhỏ có thể xong quá nhanh; có thể dùng failure_probe.py kèm theo.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 24

WordCount RDD và DataFrame trên Docker

File nộp: docker-compose.yml, data/documents/doc1..5.txt, apps/wordcount.py. Dùng cùng pipeline ở Bài 13/21 để đáp ứng đầy đủ 10 yêu cầu; thư mục kết quả thực chạy gồm rdd, dataframe, comparison.

Cả hai API cùng chuẩn hóa chữ thường, loại dấu câu và bỏ token rỗng. assert so sánh toàn bộ từ, không chỉ top 10. CSV kết quả đối chiếu ở reference_results/bai24_wordcount.csv; chưa có CSV Spark hoặc ảnh Master thực chạy.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

BÀI 25

Hệ thống dữ liệu cửa hàng

data/store_sales.csv gồm 125 dòng: 120 đơn hợp lệ, 4 dòng lỗi chủ động (ngày sai, số lượng âm, khách trống, giá không phải số), 1 dòng trùng order_id. Dữ liệu có đủ mã đơn, ngày, khách, sản phẩm, danh mục, số lượng, đơn giá.

exercises.py --bai 25 cast kiểu, parse ngày, kiểm tra null/rỗng/số dương, tách dòng lỗi, dropDuplicates theo mã đơn. Bộ dữ liệu chỉ có bản sao hoàn toàn; dữ liệu thực tế có mã đơn trùng nhưng giá trị khác cần quy tắc chọn bản ghi thay vì dropDuplicates tùy ý.

Đối chiếu: giữ 120 dòng; doanh thu toàn cửa hàng 1.164.000.000 đồng. Laptop 900.000.000; Monitor 180.000.000; Headphone 42.000.000; Keyboard 30.000.000; Mouse 12.000.000. Electronics 1.080.000.000; Accessories 84.000.000.

Số lượng mỗi sản phẩm đều 60 nên đồng hạng bán chạy. Chương trình lấy tên tăng dần khi hòa, kết quả Headphone; cần báo cáo rõ đồng hạng. So sánh tổng doanh thu sản phẩm bằng RDD reduceByKey và DataFrame groupBy/sum, assert bằng nhau.

Spark phù hợp ETL phân tán với dữ liệu vượt khả năng một máy; bộ 125 dòng chủ yếu dùng kiểm tra tính đúng. Chưa thể kết luận hiệu năng hay khả năng chịu lỗi thực nghiệm từ bộ mẫu này.

Minh chứng thực thi: xem README.md và evidence/STATUS.txt.

Tài liệu và kiểm tra bài nộp

Đề gốc: Bài 01_Spark_MasterWorker_Docker_Win.pdf, 21 trang. Phần mục tiêu có nhắc SQL, Dataset, NoSQL, Streaming và MLlib; danh sách 25 bài tập tập trung RDD, DataFrame và Docker. Hồ sơ giải đầy đủ 25 bài tập được giao.

Tài liệu chính thức Apache Spark 3.5.1:

https://archive.apache.org/dist/spark/docs/3.5.1/spark-standalone.html

https://archive.apache.org/dist/spark/docs/3.5.1/submitting-applications.html

Kiểm tra trước nộp: chạy Run-All.ps1, lưu notebook có output, thêm ảnh Master và terminal, kiểm tra CSV và đọc transcript. Các số đo thời gian và ảnh thực nghiệm không được thay bằng số liệu minh họa.